

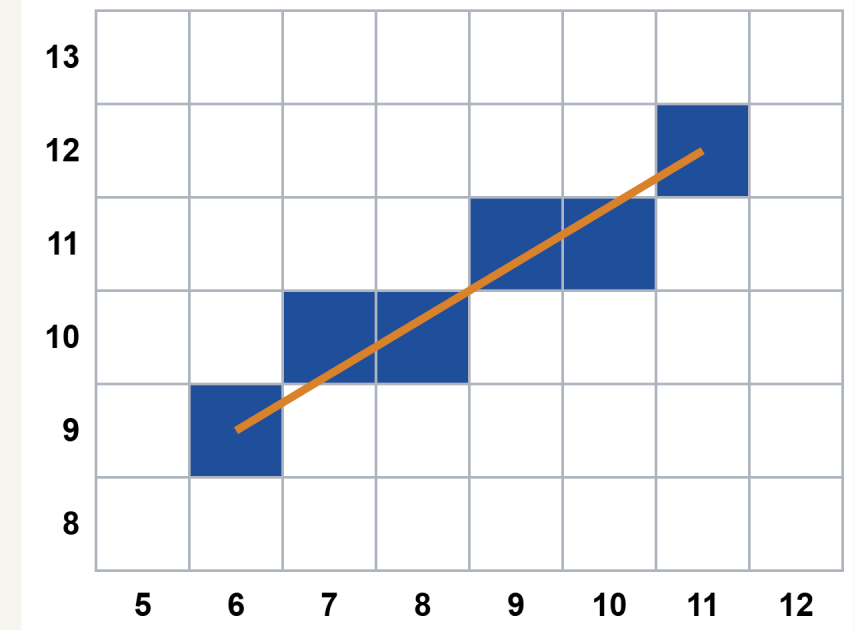
0613-403 • Week 2 • Lecture 3

Line Drawing Algorithms: DDA and Bresenham

Ekramul Haque Amin

Lecturer, AI and Data Science

Dhaka International University



Warm-up: three quick questions

Q1

What is the RGB value of yellow?

Q2

Scan convert the point (5.6, 2.2) by rounding.

Q3

Why does a steep line get gaps with the direct method?

Take one minute. Write your answers, then compare with your neighbour.

Recap: the direct method

How it works

For each x , compute $y = mx + b$ and round it.

Line (2, 3) to (7, 6): pixels (2,3) (3,4) (4,4) (5,5)
(6,5) (7,6)

Its two problems

- Slow: one multiplication for every pixel
- Steep lines ($|m| > 1$) get gaps

Today: the DDA and Bresenham algorithms fix both problems.

What is DDA?

Digital **D**ifferential **A**nalyzer: an incremental line algorithm.



Incremental

Each new point is found from the previous one, not from scratch.



Small equal steps

Move one pixel at a time along the longer direction.



Additions only

Inside the loop there is no multiplication, only adding.

The key idea: reuse the previous y

At step i: $y_i = m \cdot x_i + b$

Next step: $y_{i+1} = m \cdot x_{i+1} + b$

$= m(x_i + \Delta x) + b$

$= (m \cdot x_i + b) + m \cdot \Delta x$

$= y_i + m \cdot \Delta x$

If $\Delta x = 1$: $y_{i+1} = y_i + m$

Each new y is the old y plus the slope. One addition replaces a multiplication.

Example: $m = 0.6$, start $y = 9$

$9 \rightarrow 9.6 \rightarrow 10.2 \rightarrow 10.8 \rightarrow \dots$

Two cases, depending on the slope

Gentle slope: $|m| \leq 1$

x changes faster, so step along x.

$$x_{i+1} = x_i + 1$$

$$y_{i+1} = y_i + m$$

Every column gets exactly one pixel.

Steep slope: $|m| > 1$

y changes faster, so step along y.

$$y_{i+1} = y_i + 1$$

$$x_{i+1} = x_i + 1/m$$

Every row gets exactly one pixel: no gaps.

Why $1/m$? From $m = \Delta y / \Delta x$, with $\Delta y = 1$ we get $\Delta x = 1/m$.

The DDA algorithm (all slopes in one)

1. Input end points (x_1, y_1) and (x_2, y_2) .
2. $dx = x_2 - x_1$, $dy = y_2 - y_1$
3. $steps = \max(|dx|, |dy|)$
4. $xInc = dx \div steps$, $yInc = dy \div steps$
5. Start at $x = x_1$, $y = y_1$.
6. Repeat $steps + 1$ times: plot $(\text{round}(x), \text{round}(y))$; then $x = x + xInc$, $y = y + yInc$.

Why this works

$steps$ = the longer side, so one of $xInc$ or $yInc$ is always ± 1 and the other is the slope (or $1/\text{slope}$).

It handles gentle, steep and negative slopes without separate cases.

How to solve a DDA question in the exam

1. Write dx , dy and $steps = \max(|dx|, |dy|)$.
2. Write $xInc = dx \div steps$ and $yInc = dy \div steps$.
3. Draw a table: Step, x, y, Pixel.
4. Start from $(x1, y1)$; add $xInc$ and $yInc$ on each row.
5. Round each x and y with $\text{floor}(v + 0.5)$.
6. List the pixels, and plot them on a grid if asked.

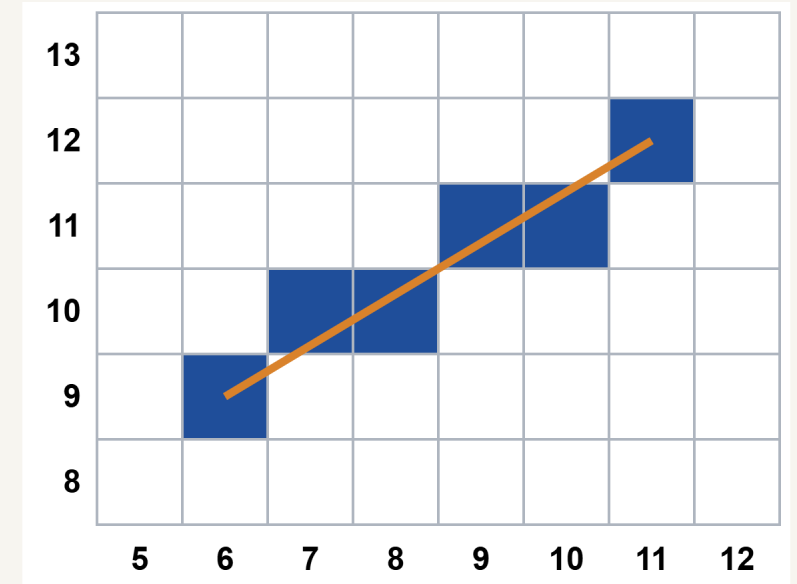
Check your answer

- There are $steps + 1$ pixels
- The first pixel is $(x1, y1)$
- The last pixel is $(x2, y2)$
- One coordinate changes by exactly 1 each row

Example 1: line from (6, 9) to (11, 12)

$dx = 5$, $dy = 3$, $steps = 5$, $xInc = 1$, $yInc = 0.6$

Step	x	y	Pixel (round x, round y)
0	6	9	(6, 9)
1	7	9.6	(7, 10)
2	8	10.2	(8, 10)
3	9	10.8	(9, 11)
4	10	11.4	(10, 11)
5	11	12	(11, 12)

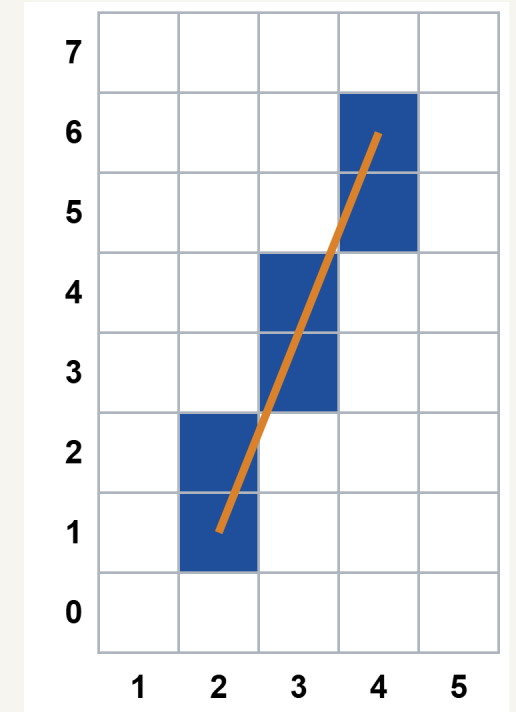


Example 2: a steep line, (2, 1) to (4, 6)

$dx = 2$, $dy = 5$, $steps = 5$, $xInc = 0.4$, $yInc = 1$

$m = 2.5 > 1$, so y goes up by 1 each step.

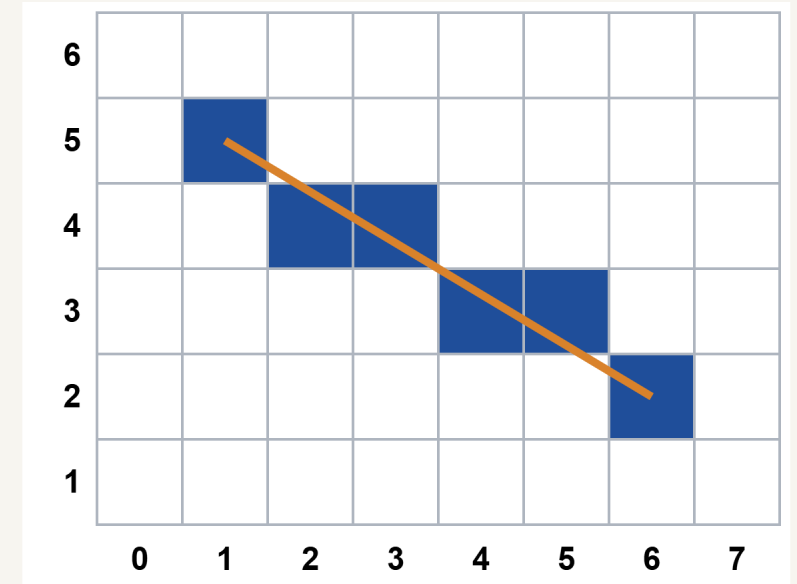
Step	x	y	Pixel (round x, round y)
0	2	1	(2, 1)
1	2.4	2	(2, 2)
2	2.8	3	(3, 3)
3	3.2	4	(3, 4)
4	3.6	5	(4, 5)
5	4	6	(4, 6)



Example 3: negative slope, (1, 5) to (6, 2)

$dx = 5$, $dy = -3$, $steps = 5$, $xInc = 1$, $yInc = -0.6$

Step	x	y	Pixel (round x, round y)
0	1	5	(1, 5)
1	2	4.4	(2, 4)
2	3	3.8	(3, 4)
3	4	3.2	(4, 3)
4	5	2.6	(5, 3)
5	6	2	(6, 2)



Special cases: DDA still works

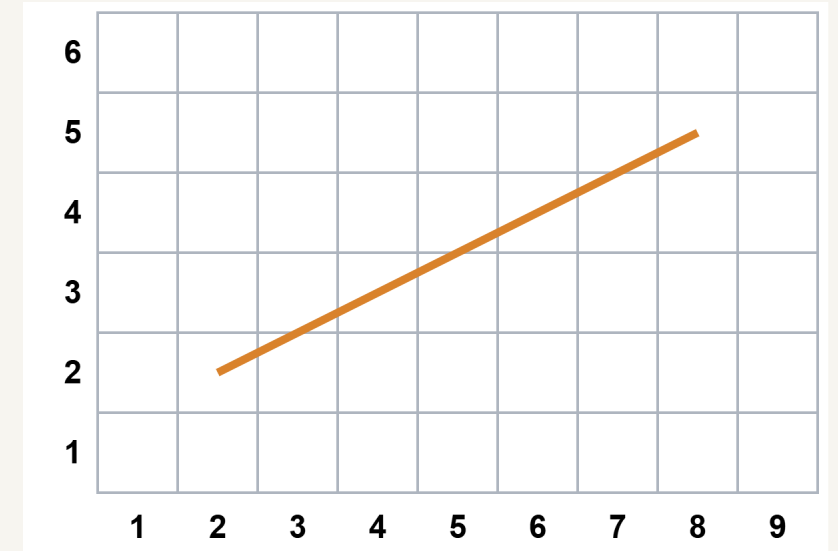
Case	Example	xInc, yInc	What happens
Horizontal line	(2, 3) to (7, 3)	1, 0	y never changes
Vertical line	(4, 1) to (4, 6)	0, 1	x never changes
45° diagonal	(1, 1) to (5, 5)	1, 1	No rounding needed
Right to left	(8, 5) to (2, 2)	-1, -0.5	Same pixels as (2, 2) to (8, 5)
Single point	(3, 3) to (3, 3)	steps = 0	Plot one pixel; do not divide by 0

Class exercise: your turn

Use DDA to scan convert the line from (2, 2) to (8, 5).

1. Find dx , dy , steps, $xInc$ and $yInc$
2. Fill in the table
3. Shade the pixels on the grid

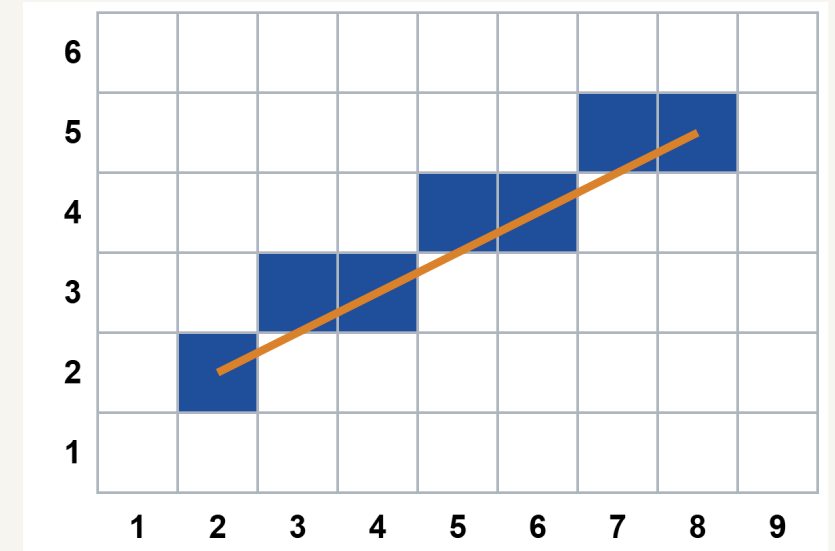
Time: 5 minutes. Work in pairs.



Exercise answer: (2, 2) to (8, 5)

$dx = 6$, $dy = 3$, $steps = 6$, $xInc = 1$, $yInc = 0.5$

Step	x	y	Pixel (round x, round y)
0	2	2	(2, 2)
1	3	2.5	(3, 3)
2	4	3	(4, 3)
3	5	3.5	(5, 4)
4	6	4	(6, 4)
5	7	4.5	(7, 5)
6	8	5	(8, 5)



DDA in C++ with OpenGL

```
void ddaLine(int x1, int y1, int x2, int y2) {  
    int dx = x2 - x1, dy = y2 - y1;  
    int steps = max(abs(dx), abs(dy));  
    float xInc = dx / (float) steps;  
    float yInc = dy / (float) steps;  
    float x = x1, y = y1;  
    glBegin(GL_POINTS);  
    for (int i = 0; i ≤ steps; i++) {  
        glVertex2i(round(x), round(y));  
        x += xInc; y += yInc;  
    }  
    glEnd();  
}
```

Advantages and disadvantages



Advantages

- Faster than the direct method: no multiplication in the loop
- Simple to understand and code
- Works for every slope, with no gaps



Disadvantages

- Uses floating-point numbers, slower than integers
- Rounding at every step takes time
- Small errors add up on long lines, so pixels can drift

Part 2: Bresenham's algorithm uses only integers and fixes these problems.

Direct method vs DDA

	Direct method	DDA
How y is found	$y = mx + b$ every time	$y = \text{previous } y + m$
Work per pixel	Multiply, add, round	Add, round
Steep lines	Gaps unless handled separately	No gaps: steps along the longer side
Number type	Floating point	Floating point
Speed	Slow	Faster

Part 2

Bresenham's line algorithm

Choosing pixels with whole numbers only

Why another line algorithm?

DDA

Adds 0.6, 0.6, 0.6 ... and rounds every time.

- Floating-point numbers
- Rounding at every step
- Small errors can build up

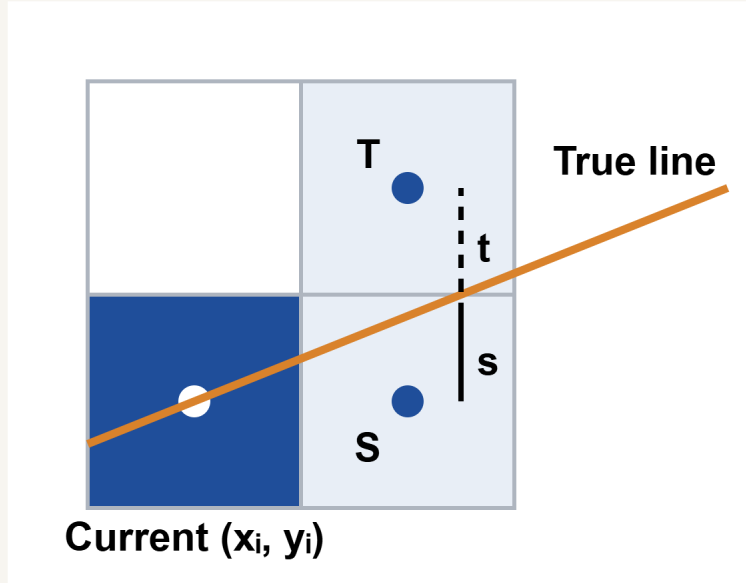
Bresenham (1965)

Decides the next pixel with a whole-number test.

- Only integer addition and subtraction
- No rounding at all
- Used inside graphics hardware

Idea: at each step there are only two possible pixels. Pick the one closer to the true line.

Only two choices at each step



For a line with slope $0 \leq m \leq 1$, x always goes up by 1. The next pixel is either:

$$S = (x_i + 1, y_i)$$

the pixel to the right (y stays the same)

$$T = (x_i + 1, y_i + 1)$$

the pixel to the right and one up

s and t are the distances from S and T to the true line. Choose the smaller one.

The decision parameter

True line at $x_i + 1$: $y = m(x_i + 1) + b$

$s = y - y_i$, $t = (y_i + 1) - y$

$s - t = 2y - 2y_i - 1$

Multiply by dx to remove the fraction $m = dy/dx$:

$$d_i = dx(s - t) = 2dy \cdot x_i - 2dx \cdot y_i + c$$

$d_i < 0$

$s < t$: S is closer. Keep y .

$d_i \geq 0$

$s \geq t$: T is closer. Add 1 to y .

Updating d with integers only

Start

$$d_1 = 2dy - dx$$

From the first end point

If $d < 0$ (choose S)

$$d = d + 2dy$$

y stays the same

If $d \geq 0$ (choose T)

$$d = d + 2(dy - dx)$$

y goes up by 1

$2dy$ and $2(dy - dx)$ are fixed numbers, worked out once. Inside the loop there is only one addition and one comparison.

Bresenham's algorithm ($0 \leq m \leq 1$)

1. Input (x_1, y_1) and (x_2, y_2) , with $x_1 < x_2$.
2. $dx = x_2 - x_1$, $dy = y_2 - y_1$
3. $d = 2dy - dx$; $inc1 = 2dy$; $inc2 = 2(dy - dx)$
4. Plot (x, y) , starting at (x_1, y_1) .
5. If $d < 0$: $d = d + inc1$. Else: $d = d + inc2$ and $y = y + 1$.
6. $x = x + 1$. Repeat from step 4 until $x = x_2$.

Exam table

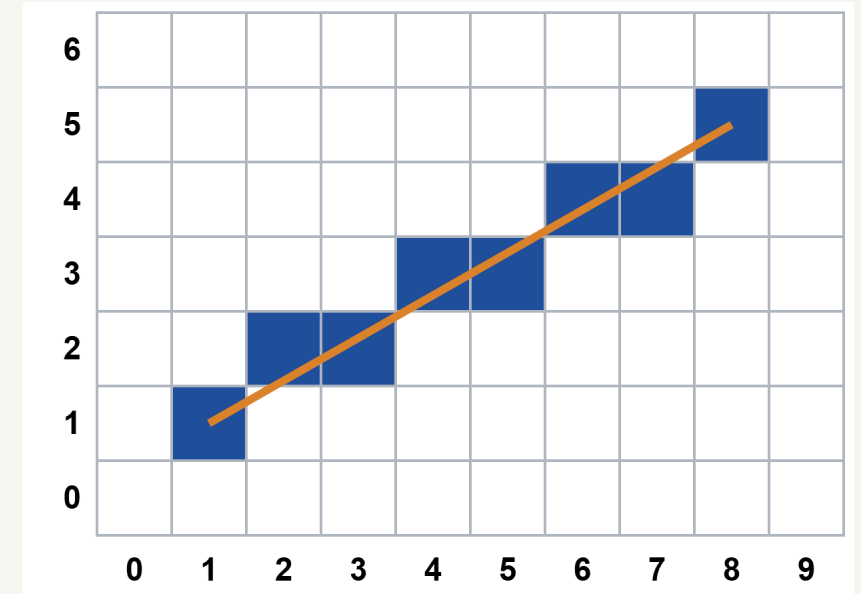
Columns: x, y, d, decision, new d.

Check: the last pixel must be (x_2, y_2) .

Example: line from (1, 1) to (8, 5)

$dx = 7$, $dy = 4$, $d_1 = 2(4) - 7 = 1$, $inc1 = 8$, $inc2 = -6$

x	y	d	Decision	New d
1	1	1	$d \geq 0 \rightarrow T: y + 1$	-5
2	2	-5	$d < 0 \rightarrow S: y \text{ stays}$	3
3	2	3	$d \geq 0 \rightarrow T: y + 1$	-3
4	3	-3	$d < 0 \rightarrow S: y \text{ stays}$	5
5	3	5	$d \geq 0 \rightarrow T: y + 1$	-1
6	4	-1	$d < 0 \rightarrow S: y \text{ stays}$	7
7	4	7	$d \geq 0 \rightarrow T: y + 1$	1
8	5	—	Stop	—



Bresenham in C++ with OpenGL

```
void bresenhamLine(int x1, int y1, int x2, int y2) {  
    int dx = x2 - x1, dy = y2 - y1;  
    int d = 2 * dy - dx;  
    int inc1 = 2 * dy, inc2 = 2 * (dy - dx);  
    int x = x1, y = y1;  
    glBegin(GL_POINTS);  
    while (x ≤ x2) {  
        glVertex2i(x, y);  
        if (d < 0) d += inc1;  
        else { d += inc2; y++; }  
        x++;  
    }  
    glEnd();  
}
```

Other slopes: same idea, small changes

Case	What to change
$x_1 > x_2$ (drawn right to left)	Swap the two end points first
Steep line, $m > 1$	Swap the roles of x and y: step y by 1, decide on x
Negative slope, $-1 \leq m < 0$	Use $ dy $, and step y by -1 instead of $+1$
Horizontal or vertical line	No decision needed: just step one coordinate

DDA vs Bresenham

	DDA	Bresenham
Numbers used	Floating point	Integers only
Work per pixel	2 additions + 2 roundings	1 addition + 1 comparison
Rounding	Every step	None
Speed	Slower	Faster
Accuracy on long lines	Errors can build up	Exact
Used in hardware	Rarely	Yes

Key points from today

1. DDA is incremental: $y_{i+1} = y_i + m$ when x steps by 1.
2. Step along the longer direction: $\text{steps} = \max(|dx|, |dy|)$.
3. $x\text{Inc} = dx \div \text{steps}$ and $y\text{Inc} = dy \div \text{steps}$; round each point.
4. It works for gentle, steep and negative slopes.
5. It still uses floating point and rounding.
6. Bresenham picks S or T with a decision parameter d , starting at $d = 2dy - dx$.
7. $d < 0$: add $2dy$. $d \geq 0$: add $2(dy - dx)$ and step y . Integers only.

Practice questions

1. Use DDA to find the pixels of the line from (1, 1) to (6, 4).
2. Use DDA to find the pixels of the line from (3, 2) to (5, 8).
3. Use DDA to find the pixels of the line from (5, 1) to (2, 7).
4. Why is steps chosen as $\max(|dx|, |dy|)$?
5. Why is DDA faster than the direct method? Give two of its drawbacks.
6. Use Bresenham's algorithm to find the pixels of the line from (20, 10) to (30, 18).
7. Use Bresenham's algorithm to find the pixels of the line from (2, 3) to (10, 6).
8. Compare DDA and Bresenham in three points.

Coming up

Next lecture

Circles and filling

Midpoint circle algorithm, 8-way symmetry, boundary fill and flood fill

This week's lab

Lab 2: DDA and Bresenham

Code both algorithms in C++/OpenGL and compare the lines they draw

Questions?

Thank you

Ekramul Haque Amin · Lecturer, AI and Data Science